# React Native with TypeScript: Masters-Level Tutorial

This pairs the internals from the masters tutorial (JSI, Fabric, Metro, threading, monorepos) with the type-system machinery that keeps those internals safe at scale: codegen-driven types, typed native modules via Nitro, project-reference monorepos, and type-level testing. Assumes comfort with both the masters and advanced-TypeScript material already.

## 1. TurboModules Are TypeScript-First by Design

Unlike the raw JSI/C++ example in the masters tutorial, the standard TurboModule path starts from a **TypeScript spec file**, and codegen derives the C++/Java/ObjC bridging code from it — TypeScript is the source of truth, not an afterthought layered on top.

```
// specs/NativeMyMath.ts
import type { TurboModule } from 'react-native';
import { TurboModuleRegistry } from 'react-
native';

export interface Spec extends TurboModule {
  fibonacci(n: number): number;
  getBatteryLevelAsync(): Promise<number>;
  readonly getConstants: () => { deviceModel:
```

```
string };
}

export default
TurboModuleRegistry.getEnforcing<Spec>
('MyMath');
```

Codegen (run via `npx react-native codegen` or automatically during builds) reads this file and generates:

- A C++ abstract base class with matching method signatures
- Java/Kotlin interface stubs (Android)
- Objective-C++ protocol declarations (iOS)

Your native implementation then **must** satisfy that generated interface — if you change `fibonacci(n: number): number` to return a `string`, the native build fails to compile until you update the native side too. This is the real value: the type contract isn't just checked in JS, it propagates into a compile-time check on native code.

## 2. Fully Typed Native Modules with Nitro

**Nitro Modules** (a newer alternative to raw TurboModule specs) pushes this further — you write a plain TypeScript interface and Nitro's codegen produces fully typed, JSI-backed C++ bindings directly, without the older codegen's ObjC/Java intermediate layer.

```
// specs/MathModule.nitro.ts
export interface MathModule {
  fibonacci(n: number): number;
```

```
  multiplyAsync(a: number, b: number):
Promise<number>;
  onValueChanged: (callback: (value: number) =>
void) => void;
}
```

```
npx nitro-codegen
```

This generates a C++ `HybridObject` base class matching the interface exactly, plus platform-specific Swift/Kotlin scaffolding. The generated JS import is fully typed end to end:

```
import { NitroModules } from 'react-native-
nitro-modules';
import type { MathModule } from
'./specs/MathModule.nitro';

const MathModule =
NitroModules.createHybridObject<MathModule>
('MathModule');

const result: number = MathModule.fibonacci(10);
// typed, synchronous, JSI-direct
```

The practical win over hand-written JSI (section 2 of the masters tutorial): you get the same zero-serialization JSI performance, but the type contract is enforced by codegen rather than by careful manual matching between a `.h` file and a hand-written `.d.ts`.

## 3. Typing Fabric Component Props Precisely

Codegen for Fabric components (introduced in the masters tutorial) supports a constrained type vocabulary — not arbitrary TypeScript. Knowing the boundaries avoids spec files that look valid to `tsc` but fail codegen:

```
import type { ViewProps, HostComponent } from
'react-native';
import type {
  Int32,
  Double,
  WithDefault,
  BubblingEventHandler,
} from 'react-
native/Libraries/Types/CodegenTypes';
import codegenNativeComponent from 'react-
native/Libraries/Utilities/codegenNativeComponen
t';

export interface NativeProps extends ViewProps {
  value: Int32;
  progress: Double;
  variant?: WithDefault<'circular' | 'linear',
'circular'>; // union + default, codegen-
supported
  onValueSettled?: BubblingEventHandler<{
finalValue: Int32 }>; // typed native event
}

export default
codegenNativeComponent<NativeProps>('MyGauge')
as HostComponent<NativeProps>;
```

- `Int32` / `Double` aren't just documentation — codegen uses them to pick the exact native numeric type, avoiding silent precision loss between JS numbers and native `int` / `float`.
- `WithDefault<Union, Default>` is how you express enum-like props with a native-side default, since plain TS union defaults ( `= 'circular'` ) aren't visible to codegen.
- `BubblingEventHandler` vs `DirectEventHandler` determines whether the event bubbles through the responder chain — picking the wrong one is a common source of gesture conflicts in nested touchable components.

## 4. Branded Types Across the Native Boundary

IDs and identifiers crossing the JS/native boundary are a common source of bugs that TypeScript's structural typing won't catch by default — a `productId: string` and a `userId: string` are interchangeable to the compiler even though mixing them up is a real bug. **Branded types** close this gap:

```
type Brand<T, B extends string> = T & { readonly
__brand: B };

type ProductId = Brand<string, 'ProductId'>;
type UserId = Brand<string, 'UserId'>;

function toProductId(id: string): ProductId {
  return id as ProductId;
```

```
}

// Native module boundary — the brand forces
callers to be explicit about which ID they have
interface CartModule {
  addToCart(productId: ProductId, userId:
UserId): Promise<void>;
}

declare const cart: CartModule;

const pid = toProductId('p_123');
// cart.addToCart(someUserId, pid); // compile
error — arguments swapped, caught here
cart.addToCart(pid, toProductId('u_456') as
unknown as UserId); // explicit, deliberate
```

This costs a small amount of ceremony at construction time in exchange for catching argument-order mistakes at native module call sites — exactly where a runtime bug would otherwise surface as a confusing native-side crash or silent no-op.

## 5. Strict Monorepo Type-Checking with Project References

In a monorepo (per the masters tutorial's Nx/Turborepo section), naive `tsc` runs re-check every package's dependencies from source on every build. **Project references** let each package build once and expose only its `.d.ts` output to consumers, dramatically speeding up type checking as the graph grows.

```
// packages/api-client/tsconfig.json
{
  "compilerOptions": {
    "composite": true,
    "declaration": true,
    "declarationMap": true,
    "outDir": "dist"
  }
}
```

```
// apps/mobile-app/tsconfig.json
{
  "references": [
    { "path": "../../packages/api-client" },
    { "path": "../../packages/ui-components" }
  ],
  "compilerOptions": { "composite": false }
}
```

```
npx tsc --build apps/mobile-app   # only
rebuilds changed packages and their dependents
```

Pair this with Turborepo's caching (`"dependsOn": ["^build"]`) and CI-wide type check times on large monorepos drop from minutes to seconds on incremental changes, since unchanged packages' type information is read from cached `.d.ts` files rather than re-inferred from source.

## 6. Typing Module Federation Remotes (Re.Pack)

Module Federation (masters tutorial section 6) has an inherent typing problem: the host app imports a remote module (`checkout/CheckoutScreen`) that doesn't exist in its own dependency graph at type-check time. The fix is a generated ambient declaration file, kept in sync via CI:

```
// types/remotes.d.ts (generated, checked into
the host app or synced via a script)
declare module 'checkout/CheckoutScreen' {
  import type { ComponentType } from 'react';
  interface CheckoutScreenProps {
    orderId: string;
    onComplete: (result: { success: boolean })
=> void;
  }
  const CheckoutScreen:
ComponentType<CheckoutScreenProps>;
  export default CheckoutScreen;
}
```

A small script run in the remote's CI pipeline extracts its public component's prop types (via the TypeScript Compiler API or a tool like `dts-bundle-generator`) and publishes this declaration file as a versioned artifact the host consumes — so a breaking prop change in the `checkout` team's release surfaces as a type error in the host's CI, not a runtime crash in production after independent deployment.

## 7. Type-Level Testing

Runtime tests (masters + advanced-TS tutorials) don't catch regressions in your *type* definitions themselves — a generic utility type can silently stop inferring correctly while every

runtime test still passes. `tsd` or `expect-type` test types directly:

```
// useCartStore.test-d.ts
import { expectType } from 'tsd';
import { useCartStore } from './useCartStore';

const items = useCartStore((state) =>
state.items);
expectType<{ id: string; name: string; price:
number }[]>(items);

// Would fail type-checking (not just at
runtime) if total() ever returned a string:
const total = useCartStore((state) =>
state.total());
expectType<number>(total);
```

```
npx tsd
```

Run this in CI alongside `tsc --noEmit` and `jest`. It's the type-system equivalent of a snapshot test — it catches the case where your code still "works" but the inferred types consumers rely on have silently degraded to `any` or widened incorrectly, which is easy to miss in code review.

## 8. Strict Compiler Settings Worth Adopting at This Scale

```
{
  "compilerOptions": {
    "strict": true,
```

```
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true,
    "verbatimModuleSyntax": true
  }
}
```

- `noUncheckedIndexedAccess` — `array[i]` returns `T | undefined` instead of `T`, which matters constantly in RN when indexing into `FlatList` data or route params derived from arrays.
- `exactOptionalPropertyTypes` — distinguishes `{ value?: number }` (key may be absent) from `{ value: number | undefined }` (key present, value undefined) — relevant for Fabric props where native code treats "absent" and "explicitly undefined" differently.
- `verbatimModuleSyntax` — ensures type-only imports are elided from the compiled output exactly as written, avoiding Metro bundling issues where a type import accidentally pulls in a runtime module.

Adopt these incrementally on a large existing codebase (`strict` alone can surface hundreds of errors) rather than all at once — most teams enable one flag per sprint and burn down the resulting errors deliberately.

## Where This Leads

- Study Nitro Modules' and Fabric codegen's own source to see how a TypeScript interface is parsed into a C++ AST — it's the clearest example of TypeScript being used as an interface definition language (IDL) rather than just an application-code type checker.

- If you're maintaining a large native-module surface, invest in a script that diffs your `.d.ts` output between releases (similar to `api-extractor`) to catch accidental breaking changes before they reach consumers.
- Combine everything here with the pure-internals masters tutorial: the threading model, Yoga, and Metro internals don't change based on TypeScript, but every boundary you cross in that material (native modules, Fabric components, Metro config) is exactly where these typing techniques apply.